

RailVue AI — Hackathon Judge Q&A; Guide

Smart India Hackathon (SIH 2026) | Complete Viva & Technical Defense Handbook

Executive Summary & Strategy for Jury Q&A;:

During a 5–10 minute hackathon evaluation, judges test beyond surface-level UI demos to probe data integrity, feature grounding, failure resiliency, and mathematical choices. This defense guide covers **24 targeted, realistic questions** across 9 core architectural pillars. Each entry includes the exact question a panelist will ask and a clear, evidence-backed answer hint citing real system benchmarks (e.g., MAE 7.31 min vs 66 min baseline, Open-Meteo caching, dead-reckoning fallback, and time-based cross-validation).

CATEGORY 1: WEATHER DATA USAGE

3 Questions

- **Q1. Why did you choose Open-Meteo over other weather APIs like OpenWeatherMap or WeatherAPI, and how are these features injected into your model?**

Strong Defense Strategy & Answer:

Highlight that Open-Meteo is open-access with no restrictive rate-limit API keys, offers historical ERA5 reanalysis and high-resolution forecasts, and station coordinates (from stations.json) map directly to fetch rainfall (mm), temperature (°C), humidity, and wind speed.

- **Q2. What happens in production if the Open-Meteo API experiences downtime or network latency?**

Strong Defense Strategy & Answer:

Mention your graceful degradation strategy: station-wise weather is cached with TTL, and if the API drops, the pipeline automatically falls back to historical seasonal averages (climatological norms) without crashing or blocking the sub-15ms ETA inference loop.

- **Q3. Did weather features show a strong statistical correlation with delays, or did they only act as marginal noise?**

Strong Defense Strategy & Answer:

Be honest that weather has a weak linear correlation on normal sunny days, but serves as a critical non-linear trigger for systemic delays during extreme monsoon downpours and winter fog (low temperatures combined with high humidity in Dec–Feb).

CATEGORY 2: CONGESTION SCORE PROXY

2 Questions

- **Q1. Since Indian Railways doesn't publicly expose real-time track circuit or signaling block occupancy, how exactly is your congestion score derived?**

Strong Defense Strategy & Answer:

Explain the exact formula from prepare_datasets.py: $congestion_score = (delayed_trains / total_trains) \times average_delay_magnitude$, computed per route segment (station_curr_station_next), hour of day, and day of week.

- **Q2. What are the limitations of using this historical proxy, and how heavily does the model weigh it compared to physical features?**

Strong Defense Strategy & Answer:

Acknowledge that it is a trailing statistical proxy rather than live physical block occupancy. Consequently, physical distance remaining and scheduled travel time carry the primary structural weight, while the congestion score provides localized penalties at bottleneck junctions like CNB, PRYJ, and DDU.

CATEGORY 3: SYSTEM ARCHITECTURE & API RESILIENCY

3 Questions

- **Q1. Walk us through the end-to-end data flow from the passenger search in React to the FastAPI prediction response.**

Strong Defense Strategy & Answer:

Explain that the React/TypeScript frontend queries `/api/trains/{id}/live`; FastAPI matches the active track segment and remaining distance, loads the serialized bundle (`eta_production_bundle.pkl`) in memory for sub-15ms inference, and returns updated ETAs with confidence scores.

- **Q2. How does your architecture decouple live train tracking from the heavy ML prediction pipeline to handle high concurrency?**

Strong Defense Strategy & Answer:

Clarify that live telemetry updates run asynchronously via a background polling/ticker service into an in-memory/cache store, while ML predictions are purely stateless read queries that evaluate pre-computed feature vectors without blocking external API calls.

- **Q3. What happens if the external RailRadar/NTES API fails or rate-limits—and how did you resolve earlier mock-fallback issues?**

Strong Defense Strategy & Answer:

Explain that earlier unhandled API errors caused inconsistent mock fallbacks; you resolved this by implementing an explicit Dead-Reckoning engine (advancing train position based on sectional speed and elapsed timetable schedule), health status badges, and an active fallback cache.

CATEGORY 4: ML ARCHITECTURE & MODEL CHOICE

3 Questions

- **Q1. Why did a basic Linear Regression model outperform or match advanced gradient-boosted trees like XGBoost and Random Forest on this dataset?**

Strong Defense Strategy & Answer:

Explain that total remaining travel time has an overwhelmingly linear physical foundation ($time \approx distance / speed + delay$); because `scheduled_remaining_time` and `distance_remaining` are strong linear anchors, standard Linear Regression achieves high precision (MAE ~7.31 min) without overfitting.

- **Q2. What did your Stacking Regressor and hyperparameter tuning reveal about XGBoost's actual contribution?**

Strong Defense Strategy & Answer:

Point out that in the Ridge meta-learner stacking setup, the coefficient assigned to Linear Regression was ~1.0 while XGBoost was near zero for gross trip time, proving that XGBoost's true value lies in the isolated *delay-deviation task* ($\Delta y = \text{actual} - \text{scheduled}$) rather than raw journey duration.

- **Q3. Why did you avoid complex deep learning architectures like LSTMs or Transformers, and how do you justify the interpretability tradeoff?**

Strong Defense Strategy & Answer:

Emphasize tabular data efficiency, sub-millisecond CPU inference, and operational auditability: railway section controllers require transparent feature weights and SHAP explanations rather than an unexplainable black box when making train routing decisions.

CATEGORY 5: FEATURE ENGINEERING & DATA INTEGRITY**3 Questions**

- **Q1. What features were engineered versus raw telemetry, and how did you prevent time-based data leakage during training?**

Strong Defense Strategy & Answer:

Raw features were station IDs, timestamps, and current delay; engineered features included route segments, historical station/route delay means, and congestion scores. Leakage was strictly prevented by computing all historical aggregations only on training folds and using chronological TimeSeriesSplit.

- **Q2. How did you encode high-cardinality categorical features like station codes and train numbers?**

Strong Defense Strategy & Answer:

State that you used OrdinalEncoder mapping with explicit unknown-category handling and fallback to global segment averages, avoiding sparse matrix blowup from One-Hot Encoding across hundreds of railway stations.

- **Q3. How does your preprocessing handle missing or corrupted sensor values in transit?**

Strong Defense Strategy & Answer:

Mention that numeric telemetry fields use median imputation calculated strictly from training distributions, categorical fields use mode imputation, and missing delay numbers default to the scheduled timetable offset.

CATEGORY 6: ACCURACY, METRICS & BENCHMARKS**3 Questions**

- **Q1. Can you give us the exact performance numbers of your model versus a naive schedule baseline?**

Strong Defense Strategy & Answer:

Cite your concrete benchmark results: the naive schedule baseline produced an MAE of ~66 minutes (or ~18–26 mins on zero-deviation), whereas the trained regression model cut MAE down to **~7.31 minutes** (RMSE ~9.22 mins), delivering a >42% error reduction.

- **Q2. Your model reports an R^2 of ~0.998. Isn't that suspiciously high or deceptive for railway delays?**

Strong Defense Strategy & Answer:

Acknowledge this upfront: gross remaining travel time spans hundreds of minutes, which mathematically inflates R^2 ; that is why you also built and evaluated the isolated **Delay-Only Task** (Δy), where the model achieved an honest $R^2 \approx 0.50$ solely on unpredictable delay fluctuations.

- **Q3. How does the model perform across different journey lengths, particularly on long-distance trips versus short arrivals?**

Strong Defense Strategy & Answer:

Reference your distance-bucket MAE breakdown: short trips (<200 km) have the lowest absolute error (~3–4 mins), while long-distance hauls (>600 km) accumulate variance (~10–12 mins) due to downstream congestion compounding.

CATEGORY 7: NOVELTY & REAL-WORLD VIABILITY**2 Questions**

- **Q1. How is RailVue AI superior to the existing NTES or IRCTC live running status that passengers already use?**

Strong Defense Strategy & Answer:

NTES relies on static, retrospective timetable offsets (*current delay + schedule*) that assume a train will maintain its current delay forever; RailVue AI dynamically anticipates future track congestion, section bottlenecks, and weather friction ahead of the train.

- **Q2. What is the practical roadmap to deploy this prototype at Indian Railways production scale?**

Strong Defense Strategy & Answer:

Explain that the system can sit on top of the existing CRIS FOIS/COA (Control Office Application) and RTIS (locomotive GPS) data feeds via Kafka message streams, running containerized FastAPI worker pods behind a Redis caching layer for millions of passenger hits.

CATEGORY 8: FAILURE MODES & EDGE CASES**3 Questions**

- **Q1. What is your 'cold-start' strategy for a newly introduced train or an unfamiliar route with zero historical data?**

Strong Defense Strategy & Answer:

The pipeline defaults to the physical kinematic baseline (*distance / speed + current delay*) combined with broad zonal averages (e.g., Northern Railway historical averages) until dedicated trip history is accumulated.

- **Q2. How does the model react to extreme unforeseen disruptions like track maintenance, derailments, or sudden protests?**

Strong Defense Strategy & Answer:

Clarify that standard regression models cannot foresee unannounced physical track closures; to handle this, the system incorporates the **Delay-Risk Classifier** and anomaly detection thresholds to flag abnormal station dwell times and trigger manual controller overrides.

- **Q3. How do you monitor for model drift across changing seasons (e.g., monsoon to winter fog)?**

Strong Defense Strategy & Answer:

Propose a weekly automated retraining cron job comparing rolling 7-day MAE against the baseline, triggering retraining with updated seasonal weather and congestion weights when performance degrades past an operational threshold.

CATEGORY 9: TEAM & PROJECT EXECUTION**2 Questions**

- **Q1. What was the single most difficult technical bug your team had to debug during this hackathon build?**

Strong Defense Strategy & Answer:

Discuss solving the subtle data leakage where future journey delays were accidentally leaking into station historical aggregations, which initially produced unrealistically perfect metrics until strict GroupShuffleSplit and chronological splits were enforced.

- **Q2. If your team had another two weeks to work on RailVue AI, what feature would you prioritize next?**

Strong Defense Strategy & Answer:

Mention integrating real-time loco-pilot telemetry from RTIS via direct WebSocket streams and building automated rake-turnaround prediction for incoming trains at terminus platforms.

